```
`timescale 1ns / 1ps
module top (
    input wire i_clk,
    input wire i_user_btn,
    input wire i_rst,
    output wire o_pwd_valid,
    output wire o_pwd_invalid,
    output wire o_lockout,
    output wire o_clk
);

    wire enable;

    clockdiv cdiv (
        .i_clk(i_clk),
        .i_rst(i_rst),
        .o_clk(o_clk),
        .o_en (enable)
    );

    mealy_detector uut (
        .i_clk(i_clk),
        .i_rst(i_rst),
        .i_en(enable),
        .i_dat_in(i_user_btn),
        .o_pwd_valid(o_pwd_valid),
        .o_pwd_invalid(o_pwd_invalid),
        .o_lockout(o_lockout)
    );
endmodule

module clockdiv #(
    parameter DESIRED_FREQ = 1,
    parameter IP_FREQ = 125000000
) (
    input  wire i_clk,
    input  wire i_rst,
    output reg o_clk,
    output reg  o_en
);

    localparam integer SETPOINT = (IP_FREQ) / (2 * DESIRED_FREQ);
    localparam integer COUNTER_WIDTH = $clog2(SETPOINT);

    reg [COUNTER_WIDTH-1:0] counter = 'b0;

    always @(posedge i_clk) begin
        if (i_rst) begin
            counter <= 'b0;
            o_en <= 1'b0;
            o_clk <= 1'b0;
        end else if (counter == SETPOINT - 1) begin
            counter <= 'b0;
            o_clk <= ~o_clk;
            o_en <= ~o_clk;
        end else begin
            counter <= counter + 1'b1;
            o_en <= 1'b0;
        end
    end
endmodule

module mealy_detector (
    input wire i_clk,
    input wire i_rst,
    input wire i_en,
    input wire i_dat_in,

    output reg o_pwd_valid,
```

```
      output reg o_pwd_invalid,
      output reg o_lockout
  );

      localparam [3:0] actual_pwd = 4'b0011;

      localparam IDLE = 3'b000;
      localparam S1 = 3'b001;
      localparam S2 = 3'b010;
      localparam S3 = 3'b011;
      localparam LOCK = 3'b100;

      reg [2:0] state;

      reg [1:0] attempt_count;

      always @(posedge i_clk or posedge i_rst) begin
          if (i_rst) begin
              state         <= IDLE;
              attempt_count <= 2'b00;
              o_pwd_valid   <= 1'b0;
              o_pwd_invalid <= 1'b0;
              o_lockout     <= 1'b0;
          end else if (i_en) begin
              o_pwd_valid   <= 1'b0;
              o_pwd_invalid <= 1'b0;

              case (state)
                  IDLE: begin
                      if (i_dat_in == 1'b0) state <= S1;
                      else state <= IDLE;
                  end
                  S1: begin
                      if (i_dat_in == 1'b0) state <= S2;
                      else begin
                          o_pwd_invalid <= 1'b1;
                          if (attempt_count == 2'd2) begin
                              attempt_count <= 2'd3;
                              state <= LOCK;
                              o_lockout <= 1'b1;
                          end else begin
                              attempt_count <= attempt_count + 1'b1;
                              state <= IDLE;
                          end
                      end
                  end
                  S2: begin
                      if (i_dat_in == 1'b1) state <= S3;
                      else begin
                          o_pwd_invalid <= 1'b1;
                          if (attempt_count == 2'd2) begin
                              attempt_count <= 2'd3;
                              state <= LOCK;
                              o_lockout <= 1'b1;
                          end else begin
                              attempt_count <= attempt_count + 1'b1;
                              state <= IDLE;
                          end
                      end

                  end
                  S3: begin
                      if (i_dat_in == 1'b1) begin
                          state <= IDLE;
                          o_pwd_valid <= 1'b1;
                          attempt_count <= 2'b00;
                      end else begin
                          o_pwd_invalid <= 1'b1;
                          if (attempt_count == 2'd2) begin
```

```
                            attempt_count <= 2'd3;
                            state <= LOCK;
                            o_lockout <= 1'b1;
                        end else begin
                            attempt_count <= attempt_count + 1'b1;
                            state <= IDLE;
                        end
                    end
                end
                LOCK: begin
                    state <= LOCK;
                    o_lockout <= 1'b1;
                    o_pwd_valid <= 1'b0;
                    o_pwd_invalid <= 1'b1;
                end
                default: begin
                    state         <= IDLE;
                    attempt_count <= 2'b00;
                    o_pwd_valid   <= 1'b0;
                    o_pwd_invalid <= 1'b0;
                    o_lockout     <= 1'b0;
                end
            endcase
        end
    end
endmodule
```